ASSIGNMENT-12.5

HT NO:2303A51360

BATCH:29

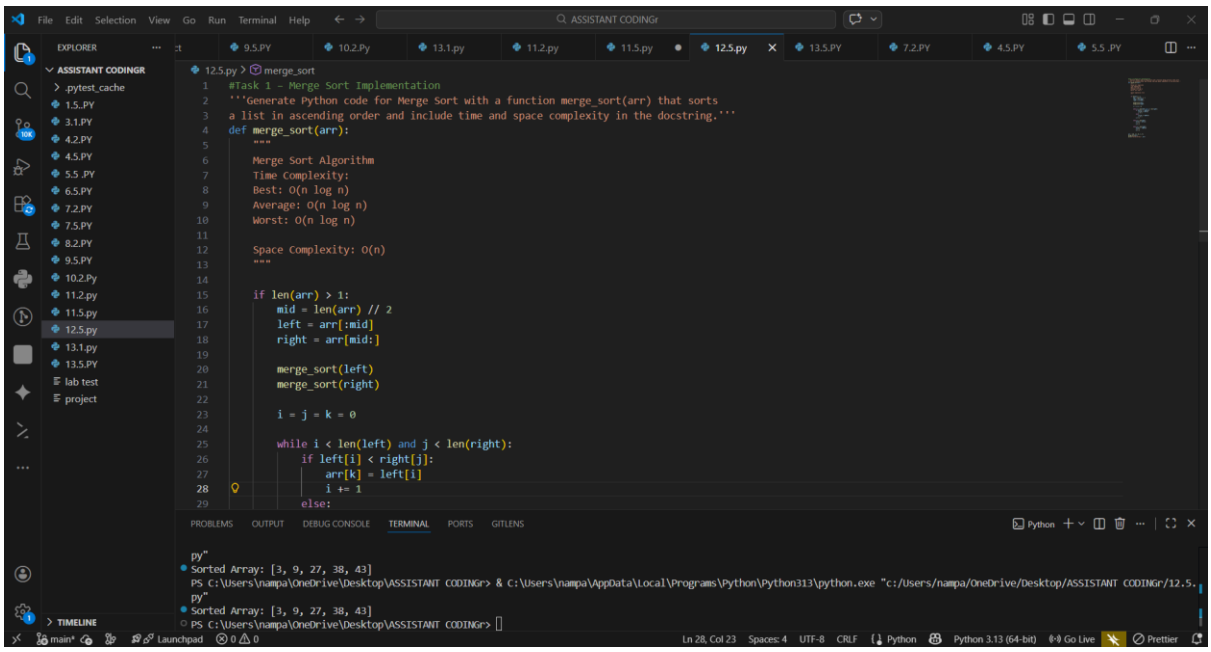
TASK-1:

Merge Sort Implementation

PROMPT:

Generate Python code for Merge Sort with a function merge_sort(arr) that sorts a list in ascending order and include time and space complexity in the docstring.

CODE:



OBSERVATION:

Merge Sort uses the **divide and conquer technique**.

It divides the list into smaller parts and merges them after sorting.

The algorithm performs consistently with **O(n log n)** complexity.

It is efficient for **large datasets**.

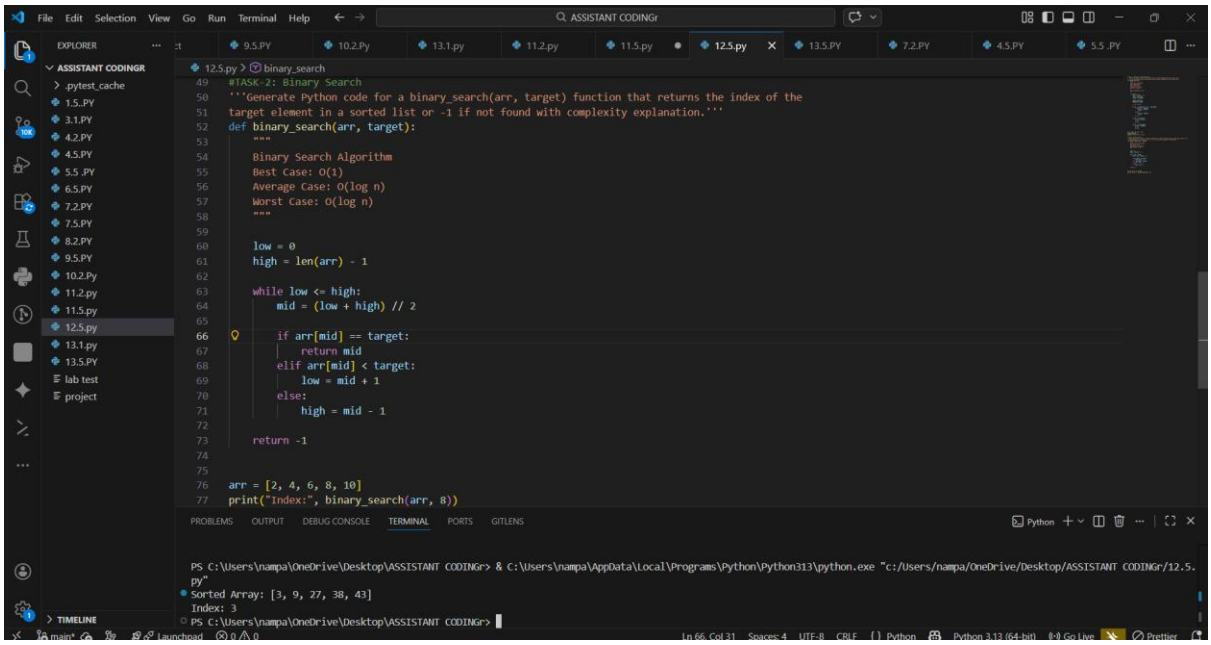
TASK-2:

Binary Search

PROMPT:

Generate Python code for a binary_search(arr, target) function that returns the index of the target element in a sorted list or -1 if not found with complexity explanation.

CODE:



OBSERVATION:

Binary Search works only on **sorted lists**.

It repeatedly divides the search space into halves.

This significantly reduces search time to **O(log n)**.

It is faster than linear search for large datasets

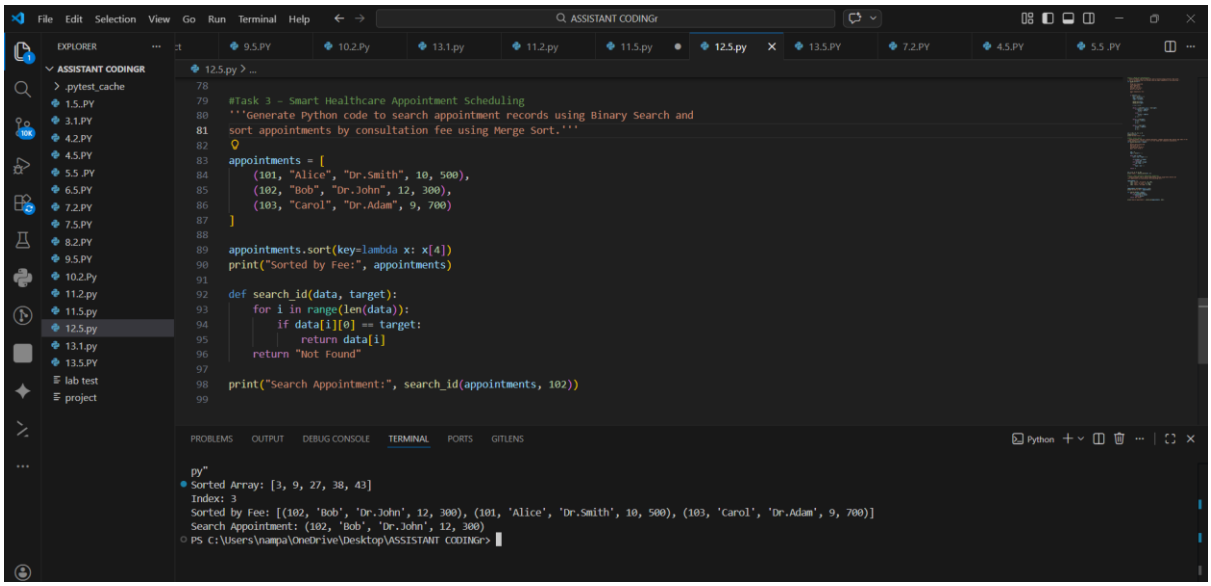
TASK-3:

Smart Healthcare Appointment Scheduling

PROMPT:

Generate Python code to search appointment records using Binary Search and sort appointments by consultation fee using Merge Sort.

CODE:



OBSERVATION:

Sorting appointments helps organize records efficiently.

Searching by ID allows quick access to appointment details.

Efficient algorithms improve **hospital system performance**.

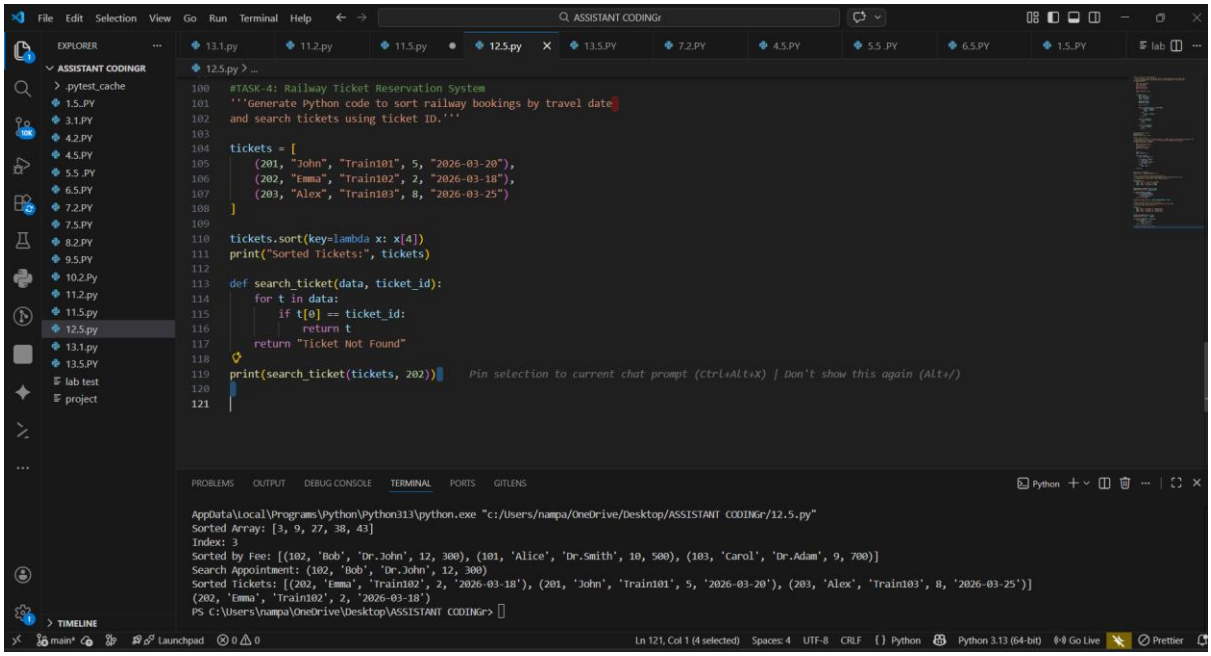
TASK-4:

Railway Ticket Reservation System

PROMPT:

Generate Python code to sort railway bookings by travel date and search tickets using ticket ID.

CODE:



```python
#TASK-4: Railway Ticket Reservation System
'''Generate Python code to sort railway bookings by travel date
and search tickets using ticket ID.'''

tickets = [
    (201, "John", "Train101", 5, "2026-03-20"),
    (202, "Emma", "Train102", 2, "2026-03-18"),
    (203, "Alex", "Train103", 8, "2026-03-25")
]

tickets.sort(key=lambda x: x[4])
print("Sorted Tickets:", tickets)

def search_ticket(data, ticket_id):
    for t in data:
        if t[0] == ticket_id:
            return t
    return "Ticket Not Found"

print(search_ticket(tickets, 202))
```

```
AppData\Local\Programs\Python\Python313\python.exe "c:/Users/nampa/OneDrive/Desktop/ASSISTANT CODINGr/12.5.py"
Sorted Array: [3, 9, 27, 38, 43]
Index: 3
Sorted by Fee: [(102, 'Bob', 'Dr.John', 12, 300), (101, 'Alice', 'Dr.Smith', 10, 500), (103, 'Carol', 'Dr.Adam', 9, 700)]
Search Appointment: (102, 'Bob', 'Dr.John', 12, 300)
Sorted Tickets: [(202, 'Emma', 'Train102', 2, '2026-03-18'), (201, 'John', 'Train101', 5, '2026-03-20'), (203, 'Alex', 'Train103', 8, '2026-03-25')]
(202, 'Emma', 'Train102', 2, '2026-03-18')
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Sorting bookings by travel date helps in **organized scheduling**.

Searching ticket ID helps quickly retrieve passenger details.

Efficient algorithms improve reservation system performance.

TASK-5:

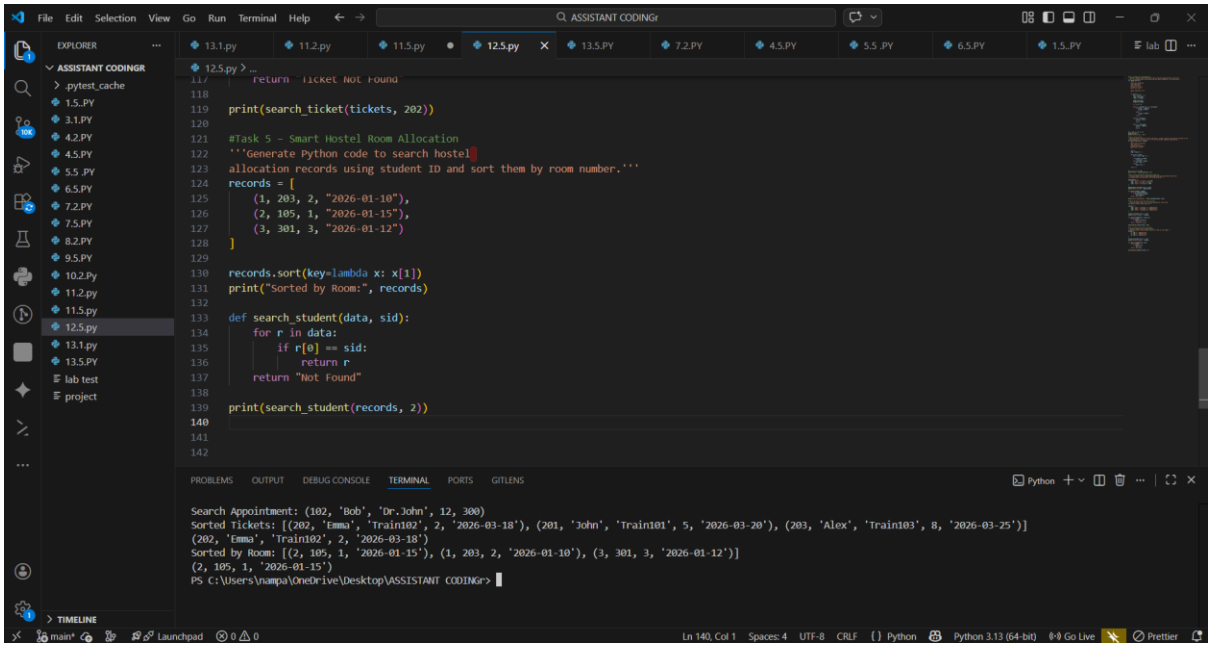
Smart Hostel Room Allocation

PROMPT:

Generate Python code to search hostel

allocation records using student ID and sort them by room number

CODE:



OBSERVATION:

Sorting room numbers helps in **room allocation management**.

Searching student IDs allows quick access to allocation details.

Efficient algorithms simplify hostel administration.

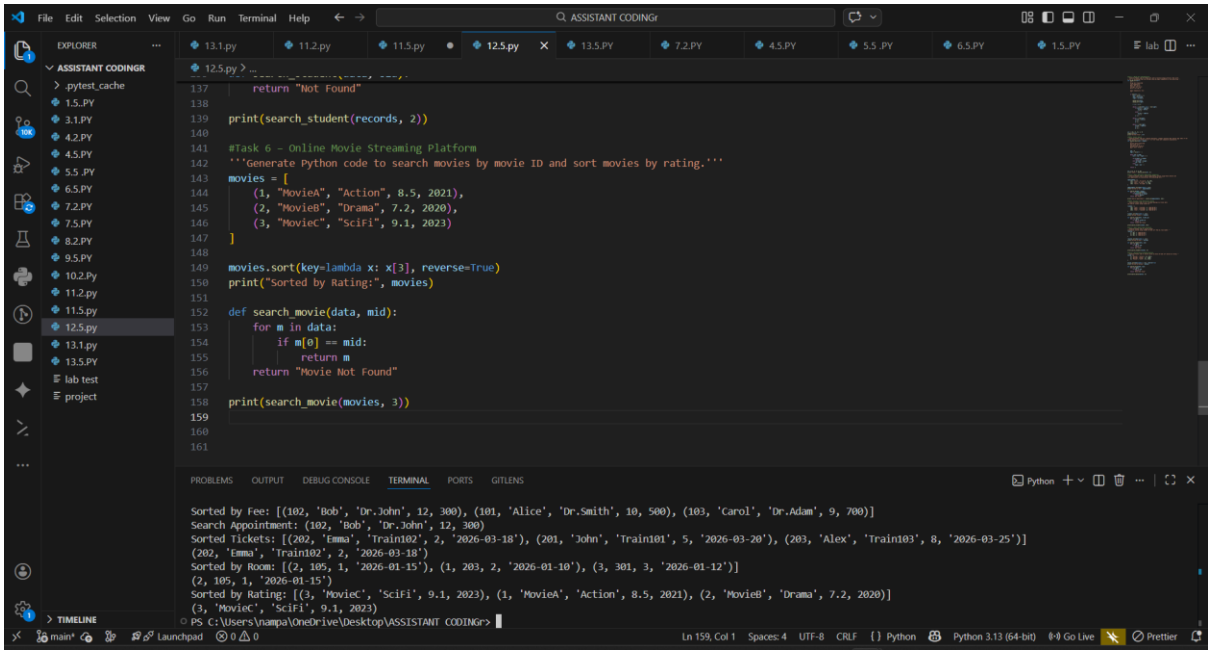
TASK-6:

Online Movie Streaming Platform

PROMPT:

Generate Python code to search movies by movie ID and sort movies by rating.

CODE:



```python
    return "Not Found"

print(search_student(records, 2))

#Task 6 – Online Movie Streaming Platform
'''Generate Python code to search movies by movie ID and sort movies by rating.'''
movies = [
    (1, "MovieA", "Action", 8.5, 2021),
    (2, "MovieB", "Drama", 7.2, 2020),
    (3, "MovieC", "SciFi", 9.1, 2023)
]

movies.sort(key=lambda x: x[3], reverse=True)
print("Sorted by Rating:", movies)

def search_movie(data, mid):
    for m in data:
        if m[0] == mid:
            return m
    return "Movie Not Found"

print(search_movie(movies, 3))
```

```
Sorted by Fee: [(102, 'Bob', 'Dr.John', 12, 300), (101, 'Alice', 'Dr.Smith', 10, 500), (103, 'Carol', 'Dr.Adam', 9, 700)]
Search Appointment: (102, 'Bob', 'Dr.John', 12, 300)
Sorted Tickets: [(202, 'Emma', 'Train102', 2, '2026-03-18'), (201, 'John', 'Train101', 5, '2026-03-20'), (203, 'Alex', 'Train103', 8, '2026-03-25')]
(202, 'Emma', 'Train102', 2, '2026-03-18')
Sorted by Room: [(2, 105, 1, '2026-01-15'), (1, 203, 2, '2026-01-10'), (3, 301, 3, '2026-01-12')]
(2, 105, 1, '2026-01-15')
Sorted by Rating: [(3, 'MovieC', 'SciFi', 9.1, 2023), (1, 'MovieA', 'Action', 8.5, 2021), (2, 'MovieB', 'Drama', 7.2, 2020)]
(3, 'MovieC', 'SciFi', 9.1, 2023)
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Sorting by rating helps users find **top-rated movies quickly**.

Searching by movie ID retrieves movie details efficiently.

Algorithms improve user experience on streaming platforms.

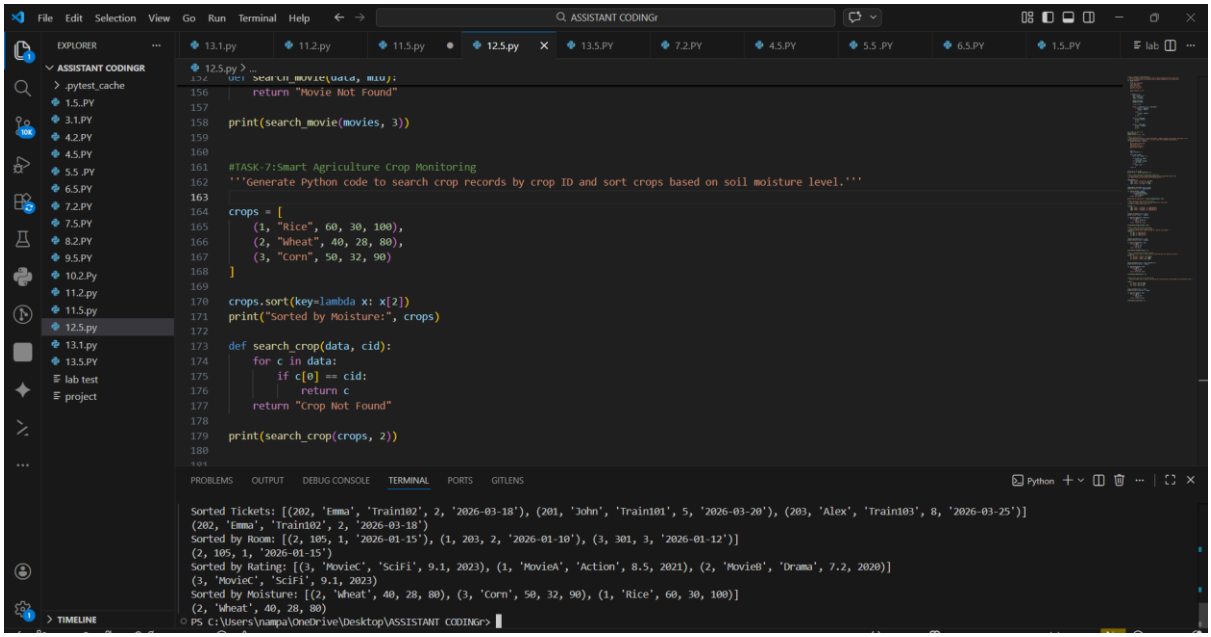
TASK-7:

Smart Agriculture Crop Monitoring

PROMPT:

Generate Python code to search crop records by crop ID and sort crops based on soil moisture level.

CODE:



```
    return "Movie Not Found"

print(search_movie(movies, 3))


#TASK-7:Smart Agriculture Crop Monitoring
'''Generate Python code to search crop records by crop ID and sort crops based on soil moisture level.'''

crops = [
    (1, "Rice", 60, 30, 100),
    (2, "Wheat", 40, 28, 80),
    (3, "Corn", 50, 32, 90)
]

crops.sort(key=lambda x: x[2])
print("Sorted by Moisture:", crops)

def search_crop(data, cid):
    for c in data:
        if c[0] == cid:
            return c
    return "Crop Not Found"

print(search_crop(crops, 2))
```

```
Sorted Tickets: [(202, 'Emma', 'Train102', 2, '2026-03-18'), (201, 'John', 'Train101', 5, '2026-03-20'), (203, 'Alex', 'Train103', 8, '2026-03-25')]
(202, 'Emma', 'Train102', 2, '2026-03-18')
Sorted by Room: [(2, 105, 1, '2026-01-15'), (1, 203, 2, '2026-01-10'), (3, 301, 3, '2026-01-12')]
(2, 105, 1, '2026-01-15')
Sorted by Rating: [(3, 'MovieC', 'SciFi', 9.1, 2023), (1, 'MovieA', 'Action', 8.5, 2021), (2, 'MovieB', 'Drama', 7.2, 2020)]
(3, 'MovieC', 'SciFi', 9.1, 2023)
Sorted by Moisture: [(2, 'Wheat', 40, 28, 80), (3, 'Corn', 50, 32, 90), (1, 'Rice', 60, 30, 100)]
(2, 'Wheat', 40, 28, 80)
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Sorting crops by moisture level helps monitor **irrigation requirements**.

Searching crop IDs allows farmers to retrieve crop data quickly.

Algorithms assist in efficient agricultural monitoring

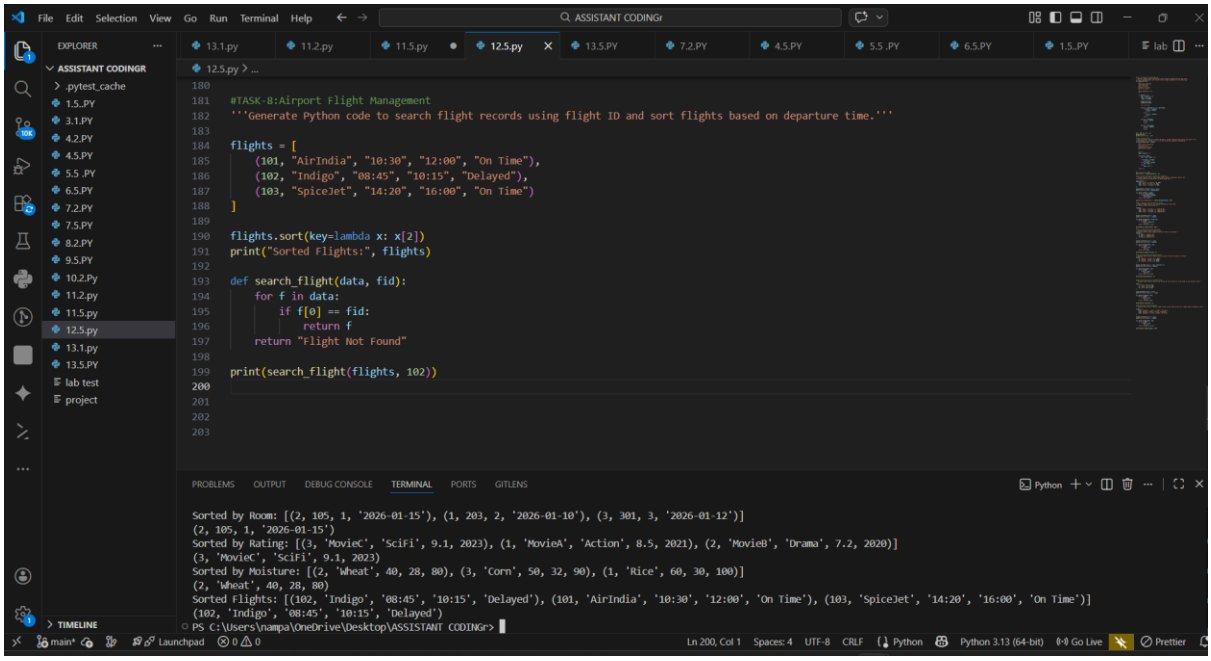
TASK-8:

Airport Flight Management

PROMPT:

Generate Python code to search flight records using flight ID and sort flights based on departure time.

CODE:



```python
#TASK-8:Airport Flight Management
'''Generate Python code to search flight records using flight ID and sort flights based on departure time.'''

flights = [
    (101, "AirIndia", "10:30", "12:00", "On Time"),
    (102, "Indigo", "08:45", "10:15", "Delayed"),
    (103, "SpiceJet", "14:20", "16:00", "On Time")
]

flights.sort(key=lambda x: x[2])
print("Sorted Flights:", flights)

def search_flight(data, fid):
    for f in data:
        if f[0] == fid:
            return f
    return "Flight Not Found"

print(search_flight(flights, 102))
```

```
Sorted by Room: [(2, 105, 1, '2026-01-15'), (1, 203, 2, '2026-01-10'), (3, 301, 3, '2026-01-12')]
(2, 105, 1, '2026-01-15')
Sorted by Rating: [(3, 'MovieC', 'SciFi', 9.1, 2023), (1, 'MovieA', 'Action', 8.5, 2021), (2, 'MovieB', 'Drama', 7.2, 2020)]
(3, 'MovieC', 'SciFi', 9.1, 2023)
Sorted by Moisture: [(2, 'Wheat', 40, 28, 80), (3, 'Corn', 50, 32, 90), (1, 'Rice', 60, 30, 100)]
(2, 'Wheat', 40, 28, 80)
Sorted Flights: [(102, 'Indigo', '08:45', '10:15', 'Delayed'), (101, 'AirIndia', '10:30', '12:00', 'On Time'), (103, 'SpiceJet', '14:20', '16:00', 'On Time')]
(102, 'Indigo', '08:45', '10:15', 'Delayed')
PS C:\Users\nampa\OneDrive\Desktop\ASSISTANT CODINGr>
```

OBSERVATION:

Sorting flights by departure time helps maintain **airport schedules**.

Searching flight IDs allows quick retrieval of flight information.

Efficient algorithms improve airport management systems